AYDT Registration Admin Portal — Full Architecture Document

Generated: 2026-03-16
Stack: Next.js 16 · React 19 · TypeScript · Tailwind CSS 4 · Supabase · PostgreSQL

Executive Summary

AYDT is a production-grade registration and administrative portal for a dance studio. The application supports:

- Public registration flow for families/dancers
- Admin dashboard for semester/class/payment management
- Email broadcast system with template rendering and delivery tracking
- Payment integration with EPG (Elavon payment gateway)
- Waitlist automation with token-based acceptance
- Advanced pricing engine supporting progressive discounts and special programs
- Family account credits system for refunds/adjustments

As of March 16, 2026, the project has completed Phase 4 (EPG payment integration).

---

1. Project Structure

```
aydt-registration-admin-portal/
├── app/
│   ├── (user-facing)/                # Public registration portal (route group)
│   │   ├── audition/[token]/         # Audition acceptance flow
│   │   ├── cart/                     # Shopping cart view
│   │   ├── error/                    # Error page
│   │   ├── family/                   # Family dashboard
│   │   ├── profile/                  # User profile management
│   │   ├── register/                 # Multi-step registration wizard
│   │   │   ├── layout.tsx
│   │   │   ├── page.tsx              # Step router
│   │   │   ├── confirmation/         # Final success page
│   │   │   ├── form/                 # Dynamic form step
│   │   │   ├── participants/         # Dancer assignment step
│   │   │   └── payment/              # Payment step
│   │   ├── semester/[id]/            # Semester detail & session browser
│   │   ├── waitlist/accept/[token]/  # Waitlist acceptance flow
│   │   └── layout.tsx
│   │
│   ├── admin/                        # Super-admin management portal
│   │   ├── _components/              # Admin layout components (TopBar.tsx)
│   │   ├── layout.tsx                # Admin shell + auth guard + sidebar nav
│   │   ├── page.tsx                  # Dashboard landing page
│   │   ├── classes/                  # Class management (CRUD)
│   │   │   ├── [id]/                 # Class detail view
│   │   │   └── page.tsx
│   │   ├── semesters/                # Semester editor (9-step wizard)
│   │   │   ├── [id]/                 # Semester detail/tabs
│   │   │   │   ├── dashboard/        # Semester stats dashboard
│   │   │   │   ├── edit/             # Edit page (loads SemesterForm)
│   │   │   │   ├── invites/          # Competition track invite management
│   │   │   │   └── page.tsx
│   │   │   ├── new/                  # New semester creation
│   │   │   │   └── discounts/        # Discount CRUD helpers
│   │   │   ├── actions/              # Server actions (all semester mutations)
```

```
│   │   │   ├── steps/                # Form step components
│   │   │   ├── SemesterForm.tsx      # Main editor orchestrator (useReducer)
│   │   │   └── page.tsx              # Semester listing
│   │   ├── emails/                   # Email broadcast management
│   │   │   ├── [id]/edit/            # Email edit page
│   │   │   ├── new/                  # New email draft
│   │   │   ├── steps/                # Email wizard steps
│   │   │   ├── actions/              # Email server actions
│   │   │   ├── EmailForm.tsx         # Email editor
│   │   │   ├── EmailsClient.tsx      # Email list (client)
│   │   │   └── page.tsx
│   │   ├── families/                 # Family records management
│   │   │   ├── [id]/page.tsx         # Family detail page
│   │   │   ├── _components/          # Family UI components
│   │   │   └── page.tsx
│   │   ├── dancers/                  # Dancer records management
│   │   ├── users/                    # User/admin management
│   │   ├── payments/                 # Payment tracking & admin adjustments
│   │   │   ├── actions/              # Mark installment paid
│   │   │   └── page.tsx
│   │   ├── sessions/                 # Session management
│   │   ├── media/                    # Media library (images)
│   │   ├── credits/                  # Family account credits management
│   │   ├── profile/                  # Admin profile settings
│   │   └── register/                 # Admin manual registration tool
│   │
│   ├── api/                          # API routes (JSON)
│   │   ├── webhooks/
│   │   │   ├── epg/route.ts          # EPG payment webhook handler
│   │   │   ├── resend/route.ts       # Resend email delivery tracking
│   │   │   └── twilio/route.ts       # SMS webhook (Phase 5)
│   │   ├── media/                    # Media CRUD endpoints
│   │   ├── register/batch-status/    # Batch status polling endpoint
│   │   ├── upload-image/route.ts     # Image upload handler
│   │   └── upload-pdf/route.ts       # PDF upload handler
│   │
│   ├── auth/                         # Authentication flows
│   │   ├── page.tsx                  # Auth landing
```

```
│   │   ├── login/page.tsx
│   │   ├── actions.ts                # signUp, login, resetPassword server actions
│   │   ├── confirm/route.ts
│   │   ├── request-password-reset/
│   │   └── reset-password/
│   │
│   ├── components/                   # Shared React components
│   │   ├── semester-flow/            # TipTap editors, form builders
│   │   ├── form/                     # Inputs, selects, etc.
│   │   ├── public/                   # SessionGrid, CartDrawer
│   │   ├── email/                    # Email preview frame
│   │   ├── media/                    # Image picker modal
│   │   └── ui/                       # Generic UI primitives
│   │
│   ├── providers/                    # React Context providers
│   │   ├── AuthProvider.tsx
│   │   ├── CartProvider.tsx          # Shopping cart (localStorage)
│   │   ├── RegistrationProvider.tsx  # Multi-step wizard state (sessionStorage)
│   │   └── SemesterDataProvider.tsx  # Semester data hydration
│   │
│   └── lib/
│       ├── actions/                  # Shared server action utilities
│       └── validation/               # Zod validation schemas
│
├── lib/
│   └── schemas/registration.ts       # Zod schemas for registration
│
├── types/
│   ├── index.ts                      # ALL domain types (~1827 lines)
│   └── public.ts                     # Public-safe type subset (~228 lines)
│
├── utils/
│   ├── supabase/
│   │   ├── client.ts                 # Browser Supabase client
│   │   ├── server.ts                 # Server Supabase client
│   │   ├── admin.ts                  # Admin/service role client
│   │   └── middleware.ts             # Auth session middleware
│   ├── payment/
```

```
│   │   └── epg.ts                    # EPG REST API client (Elavon)
│   ├── email-templates/
│   │   └── subscriptionConfirmation.ts
│   ├── tuitionEngine.ts              # Pricing calculation engine
│   ├── mapSemesterToDraft.ts         # Hydration utility for semester editor
│   ├── buildPaymentSchedule.ts       # Payment plan calculation
│   ├── detectTimeConflicts.ts        # Registration conflict checking
│   ├── ratelimit.ts                  # Upstash rate limiting
│   ├── requireAdmin.ts               # Admin auth guard
│   ├── prepareEmailHtml.ts           # Email HTML normalization
│   ├── resolveEmailVariables.ts      # Token replacement
│   ├── sendSms.ts                    # Twilio SMS integration
│   └── gradeLabel.ts                 # Grade label mapping
│
├── queries/
│   └── admin/
│       ├── index.ts                  # Admin data queries (~500 lines)
│       └── getFamilyDetail.ts        # Family detail query (new)
│
├── supabase/
│   ├── migrations/                   # 41 migration files
│   │   ├── 20260302000000_baseline.sql  (3123 lines — core schema)
│   │   ├── 20260304000000_session_pricing_and_options.sql
│   │   ├── 20260304030000_hybrid_pricing_model.sql
│   │   ├── 20260309000003_tuition_engine.sql
│   │   ├── 20260309000002_class_catalog_schema.sql
│   │   ├── 20260316000001_family_account_credits.sql
│   │   ├── 20260316000002_epg_stored_payment.sql
│   │   ├── 20260316000003_installment_charge_tracking.sql
│   │   └── 20260316000004_batch_form_data.sql
│   └── functions/                    # Deno edge functions
│       ├── process-waitlist/
│       ├── process-scheduled-emails/
│       └── send-email-broadcast/
│
├── docs/                             # Project documentation
│   ├── ARCHITECTURE_REPORT.md
│   ├── DESIGN_SYSTEM.md
```

```
│   ├── EMAIL_SYSTEM.md
│   ├── CLASS_CATALOG.md
│   ├── CLASS_SESSION_ARCHITECTURE.md
│   ├── PRICING_AND_DISCOUNTS.md
│   ├── EPG_PAYMENT_INTEGRATION.md
│   ├── EPG_WEBHOOK_ARCHITECTURE.md
│   ├── SMS_NOTIFICATIONS_ARCHITECTURE.md
│   ├── RATE_LIMITING.md
│   ├── TESTING.md
│   └── elavon/                       # Elavon REST API specs
│
├── middleware.ts                     # Auth + rate limiting
├── next.config.ts
├── tailwind.config.js
├── package.json
└── tsconfig.json
```

---

2. Dependencies (package.json)

Runtime Dependencies

Package	Version	Purpose
next	16.0.10	App framework
react / react-dom	19.2.0	UI library
@supabase/ssr	^0.7.0	Supabase SSR auth
@supabase/supabase-js	^2.80.0	Supabase client
tailwindcss	^4	CSS framework
zod	^4.1.13	Schema validation
react-hook-form	^7.67.0	Form state management
@hookform/resolvers	^5.2.2	Zod + RHF bridge
@tiptap/react	^3.20.0	Rich text editor
@tiptap/starter-kit	^3.20.0	TipTap base extensions
resend	^6.9.2	Transactional email delivery
recharts	^3.8.0	Dashboard charts
lucide-react	^0.577.0	Icons
uuid	^13.0.0	UUID generation
@upstash/ratelimit	^2.0.8	Rate limiting
@upstash/redis	^1.37.0	Redis client
twilio	^5.13.0	SMS (Phase 5)
image-size	^2.0.2	Image dimension detection
sass	^1.94.2	SCSS support

Dev Dependencies

Package	Purpose
typescript ^5	Type checking
eslint ^9	Linting
vitest ^4.0.14	Unit tests
playwright ^1.57.0	E2E tests
@testing-library/react ^16	Component testing

Scripts

```
npm run dev           # Start dev server
npm run build         # Build for production
npm run start         # Run production server
npm run type-check    # TypeScript check
npm run lint          # ESLint
npm run test          # Vitest (unit tests)
npm run test:e2e      # Playwright (e2e tests)
npm run db:new        # Create new migration
npm run db:seed       # Seed database
```

---

3. Domain Types (types/index.ts — ~1827 lines)

User

```
interface User {
  id: string
  family_id: string
  email: string
  first_name: string
  middle_name?: string | null
  last_name: string
  phone_number: string
  is_primary_parent: boolean
  role: string  // 'super_admin' | 'admin' | 'parent'
  status: string
  created_at: string
  display_name?: string | null
  signature_html?: string | null
  signature_config?: SignatureConfig | null
  reply_to_email?: string | null
  address_line1?: string | null
  city?: string | null
  state?: string | null
  zipcode?: string | null
  sms_opt_in?: boolean
  sms_verified?: boolean
}
```

Dancer

```
interface Dancer {
  id: string
  first_name: string
  middle_name?: string | null
  last_name: string
  gender: string | null
  birth_date: string | null    // "YYYY-MM-DD"
  grade: string | null
  email: string | null
  phone_number: string | null
  address_line1: string | null
  city: string | null
  state: string | null
  zipcode: string | null
  is_self: boolean
  created_at: string
  users: { id: string; first_name: string; last_name: string; email: string }[]
  registrations?: Registration[]
}
```

Family

```
interface Family {
  id: string
  family_name: string | null
  created_at: string
  users: User[]
  dancers: Dancer[]
}
```

Class & Session

```
type Discipline = 'ballet' | 'tap' | 'broadway' | 'hip_hop' | 'contemporary'
               | 'technique' | 'pointe' | 'jazz' | 'lyrical' | 'acro'
type Division = 'early_childhood' | 'junior' | 'senior' | 'competition'
type DayOfWeek = 'monday' | 'tuesday' | 'wednesday' | 'thursday'
               | 'friday' | 'saturday' | 'sunday'
type ClassVisibility = 'public' | 'hidden' | 'invite_only'
type ClassEnrollmentType = 'standard' | 'audition'

interface DanceClass {
  id: string
  semester_id: string
  name: string
  discipline: Discipline
  division: Division
  description: string | null
  min_age: number | null
  max_age: number | null
  min_grade: number | null
  max_grade: number | null
  is_active: boolean
  is_competition_track: boolean
  requires_teacher_rec: boolean
  visibility: ClassVisibility
  enrollment_type: ClassEnrollmentType
  created_at: string
  updated_at: string
  class_sessions?: ClassSession[]
}

interface ClassSession {
  id: string
  class_id: string
  semester_id: string
  day_of_week: string
  start_time: string | null    // "HH:MM:SS"
  end_time: string | null
```

```
  start_date: string | null    // "YYYY-MM-DD"
  end_date: string | null
  location: string | null
  instructor_name: string | null
  capacity: number | null
  registration_close_at: string | null
  is_active: boolean
  created_at: string
  cancelled_at?: string | null
  cancellation_reason?: string | null
  session_occurrence_dates?: SessionOccurrenceDate[]
}

interface SessionOccurrenceDate {
  id: string
  session_id: string
  date: string          // "YYYY-MM-DD"
  is_cancelled: boolean
  cancellation_reason: string | null
  created_at: string
}
```

Pricing

```
interface TuitionRateBand {
  id: string
  semester_id: string
  division: string
  weekly_class_count: number
  base_tuition: number
  progressive_discount_percent: number   // 0–100
  semester_total?: number
  autopay_installment_amount?: number
  notes?: string
  created_at: string
  updated_at: string
}

interface SemesterFeeConfig {
  semester_id: string
  registration_fee_per_child: number     // default 40.00
  family_discount_amount: number         // default 50.00
  auto_pay_admin_fee_monthly: number     // default 5.00
  auto_pay_installment_count: number     // default 5
  senior_video_fee_per_registrant: number // default 15.00
  senior_costume_fee_per_class: number   // default 65.00
  costume_fee_exempt_keys: string[]      // ['technique', 'pointe', 'competition']
  created_at: string
  updated_at: string
}

type DraftTuitionRateBand = {
  _clientKey: string              // React key for unsaved rows
  id?: string
  division: string
  weekly_class_count: number
  base_tuition: number
  progressive_discount_percent: number
  semester_total?: number
  autopay_installment_amount?: number
```

```
  notes?: string
}

type DraftSpecialProgramTuition = {
  _clientKey: string
  id?: string
  programKey: string              // 'technique' | 'pointe' | 'competition_*' |
'early_childhood'
  programLabel: string
  semesterTotal: number
  autoPayInstallmentAmount: number | null
  autoPayInstallmentCount: number | null
  registrationFeeOverride?: number | null
  notes?: string
}

type DraftCoupon = {
  _clientKey: string
  id?: string
  name: string
  code: string | null             // null = auto-apply by date range
  value: number
  valueType: 'flat' | 'percent'
  validFrom: string | null
  validUntil: string | null
  maxTotalUses: number | null
  usesCount: number
  maxPerFamily: number
  stackable: boolean
  eligibleSessionsMode: 'all' | 'selected'
  sessionIds?: string[]
  isActive: boolean
  appliesToMostExpensiveOnly?: boolean
  eligibleLineItemTypes?: Array<'tuition' | 'registration_fee' | 'recital_fee'>
}
```

Semester

```
interface Semester {
  id: string
  name: string
  description?: string | null
  status: 'draft' | 'scheduled' | 'published' | 'archived'
  registration_form: RegistrationFormElement[]   // JSONB
  confirmation_email: ClassEmailConfig           // JSONB
  waitlist_settings: WaitlistSettings            // JSONB
  payment_plan?: PaymentPlan
  publish_at?: string | null
  published_at?: string | null
  created_at: string
  updated_at: string
  created_by: string
  updated_by: string
}

type WaitlistSettings = {
  enabled: boolean
  inviteExpiryHours: number      // default 48
  stopDaysBeforeClose: number    // default 3
}

type PaymentPlan = {
  type: 'pay_in_full' | 'deposit_flat' | 'deposit_percent' | 'installments'
  depositAmount?: number | null
  depositPercent?: number | null
  installmentCount?: number | null
  dueDate?: string | null
  installments?: { number: number; amount: number; dueDate: string }[]
}
```

Registration & Payment

```
interface Registration {
  id: string
  dancer_id: string
  session_id: string
  registration_batch_id: string
  status: 'pending' | 'confirmed' | 'cancelled'
  form_data: Record<string, unknown>   // JSONB
  total_amount: number | null
  hold_expires_at: string | null       // 30-min hold after pending creation
  created_at: string
}

interface RegistrationBatch {
  id: string
  parent_id?: string
  semester_id: string
  family_id: string
  cart_snapshot: CartSnapshot[]        // JSONB
  payment_intent_id?: string
  status: 'pending' | 'confirmed' | 'failed' | 'refunded'
  tuition_total?: number
  registration_fee_total?: number
  family_discount_amount: number
  auto_pay_admin_fee_total: number
  grand_total?: number
  payment_plan_type?: string
  amount_due_now?: number
  payment_reference_id?: string
  created_at: string
  confirmed_at?: string
}

interface Payment {
  id: string
  registration_batch_id: string
  order_id?: string
```

```
  payment_session_id?: string
  transaction_id?: string
  custom_reference: string            // batchId
  amount: number
  currency: string                    // 'USD'
  state: 'initiated' | 'pending_authorization' | 'authorized' | 'captured'
       | 'settled' | 'declined' | 'voided' | 'refunded' | 'held_for_review'
  event_type?: string
  raw_notification?: Record<string, unknown>   // EPG webhook payload
  raw_transaction?: Record<string, unknown>    // Full EPG transaction
  created_at: string
  updated_at: string
}

interface BatchPaymentInstallment {
  id: string
  registration_batch_id: string
  installment_number: number
  amount: number
  due_date: string                    // ISO date
  status: 'pending' | 'scheduled' | 'charged' | 'paid' | 'failed'
  charged_at?: string | null
  paid_at?: string | null
  created_at: string
}
```

Email

```
interface Email {
  id: string
  subject: string
  body_html: string
  body_json: Record<string, unknown>   // TipTap JSON
  status: 'draft' | 'scheduled' | 'sent' | 'sending' | 'failed' | 'cancelled'
  scheduled_at?: string | null
  sent_at?: string | null
  sender_name: string
  sender_email: string
  reply_to_email?: string | null
  include_signature: boolean
  created_by_admin_id: string
  updated_by_admin_id: string
  created_at: string
  updated_at: string
  deleted_at?: string | null
}

interface EmailDelivery {
  id: string
  email_id: string
  user_id: string
  email_address: string
  resend_message_id?: string
  status: 'pending' | 'sent' | 'delivered' | 'bounced' | 'complained'
  delivered_at?: string | null
  bounced_at?: string | null
  opened_at?: string | null
  clicked_at?: string | null
  created_at: string
  updated_at: string
}
```

Waitlist

```
interface WaitlistEntry {
  id: string
  session_id: string
  user_id: string
  position: number
  status: 'waiting' | 'invited' | 'accepted' | 'expired' | 'cancelled'
  invite_token?: string
  invitation_expires_at?: string | null
  invitation_sent_at?: string | null
  accepted_at?: string | null
  created_at: string
  updated_at: string
}
```

Account Credits

```
interface FamilyAccountCredit {
  id: string
  family_id: string
  amount: number
  reason?: string | null
  issued_by_admin_id?: string | null
  source_batch_id?: string | null      // Batch that triggered the credit
  used_in_batch_id?: string | null     // Batch that consumed the credit
  used_at?: string | null
  created_at: string
  is_active: boolean
}
```

EPG Payment (Elavon)

```
interface EpgOrder {
  id: string
  href: string
  total: { amount: string; currencyCode: string }
  customReference: string | null       // batchId
}

interface EpgPaymentSession {
  id: string
  href: string
  url: string                          // Hosted payment page URL
  transaction?: string | null
  state?: string
  hostedCard?: { href: string } | null
  hostedAchPayment?: { href: string } | null
}

interface EpgTransaction {
  id: string
  href: string
  type: 'sale' | 'void' | 'refund'
  isAuthorized: boolean
  state: string
  total: { amount: string; currencyCode: string }
  customReference: string | null
  authorizationCode?: string | null
  card?: {
    maskedNumber?: string | null
    last4?: string | null
    scheme?: string | null
  } | null
}

interface EpgShopper {
  id: string
  href: string
```

```
  fullName?: string | null
  email?: string | null
  customReference?: string | null      // familyId
  createdAt: string
  modifiedAt?: string | null
}

interface EpgStoredCard {
  id: string
  href: string
  shopper: string                      // Parent shopper href
  card?: {
    maskedNumber?: string | null
    last4?: string | null
    scheme?: string | null
    expirationMonth?: number | null
    expirationYear?: number | null
  } | null
  credentialOnFileType?: string | null
  customReference?: string | null
  createdAt: string
  modifiedAt?: string | null
}

interface EpgStoredAchPayment {
  id: string
  href: string
  shopper: string
  last4?: string | null
  achAccountType?: string | null
  accountName?: string | null
  achFingerprint?: string | null
  customReference?: string | null
  createdAt: string
  modifiedAt?: string | null
}

interface InstallmentCharge {
```

```
  id: string
  installment_id: string
  payment_session_id?: string
  transaction_id?: string
  state: 'initiated' | 'pending' | 'authorized' | 'captured' | 'settled'
       | 'declined' | 'failed'
  error_reason?: string | null
  created_at: string
  updated_at: string
}
```

---

4. Database Schema (Supabase PostgreSQL — 41 migrations)

Core Tables

Table	Purpose
users	Auth + profile + address; role = super_admin / admin / parent
families	Family grouping with family_name
dancers	Student records, linked to users via join table
semesters	Enrollment periods (draft → published → archived); stores registration_form, confirmation_email, waitlist_settings as JSONB
classes	Curriculum units (discipline, division, visibility, enrollment_type)
class_sessions	Time slots within a class (day, time, capacity, instructor, dates)
session_occurrence_dates	Individual calendar dates for absence tracking
session_groups	Bundled class offerings
registrations	Student enrollments (30-min hold logic, status = pending/confirmed/cancelled)
registration_batches	Multi-registrant checkout with cart_snapshot JSONB, form_data_snapshot JSONB (NEW)
batch_payment_installments	Auto-pay schedule (installment_number, amount, due_date, status)
payments	EPG payment state + raw_notification + raw_transaction JSONB
installment_charges	Per-charge tracking for recurring auto-pay attempts
family_account_credits	Dollar credits issued by admin; linked to source/used batches
tuition_rate_bands	Progressive pricing by division + weekly class count
special_program_tuition	Fixed fees for technique/pointe/competition/early_childhood
semester_fee_config	Per-semester fee constants (reg fee, family discount, video fee, costume fee)
waitlist_entries	Capacity overflow management with token-based invite acceptance
emails	Broadcast email records with TipTap JSON body
email_recipients	Snapshotted recipient list at send time

Table	Purpose
email_deliveries	Per-recipient delivery tracking via Resend webhooks
email_subscriptions	Opt-in/opt-out tracking
discounts / semester_discounts	Global discount definitions linked to semesters
media_images	Image library with folder organization
requirement_waivers	Audition/acceptance overrides
authorized_pickups	Safe pickup tracking
class_requirements	Prerequisite/recommendation rules
semester_audit_logs	Admin action history

Key Indexes

```
registrations(session_id, dancer_id, status)
registrations(registration_batch_id)
batch_payment_installments(registration_batch_id, status)
waitlist_entries(session_id, status)
session_occurrence_dates(session_id, date)
email_recipients(email_id, user_id)
email_deliveries(email_id, user_id, status)
```

Database Functions & Triggers

```
-- Auth
handle_new_user()                    -- Auto-creates family + user on Supabase signup
is_admin_or_super()
is_super_admin()

-- Registration
check_registration_time_conflict()   -- Prevents overlapping session times
prevent_child_modification_if_semester_published()
prevent_semester_core_edit_if_published()

-- Cron Jobs (pg_cron)
expire_registration_holds_cron()     -- Releases 30-min pending holds
process_waitlist()                   -- Invites next in line (every 1 minute)
process_scheduled_emails()           -- Dispatches scheduled sends
```

---

5. Authentication & Authorization

Auth Flow (Supabase)

1. **Signup:** Email/password → Supabase Auth → `handle_new_user()` trigger creates `users` + `families` row
2. **Login:** Email + password → Supabase Auth → JWT in httpOnly cookie (via `updateSession()` in middleware)
3. **Admin Guard:** `app/admin/layout.tsx` checks `users.role === 'super_admin'` → 401 if not

Roles

Role	Access
super_admin	Full admin portal access
admin	(Future: limited admin access)
parent	User-facing portal only

---

6. API Routes & Endpoints

User-Facing Routes

```
GET  /semester/[id]              — Semester detail + session browser
GET  /register                   — Multi-step registration wizard
GET  /register/form              — Dynamic form step
GET  /register/participants      — Dancer assignment step
GET  /register/payment           — EPG hosted checkout redirect
GET  /register/confirmation      — Success page
GET  /waitlist/accept/[token]    — Token-based waitlist acceptance
GET  /family                     — Family dashboard
GET  /audition/[token]           — Audition/invite acceptance
```

Admin Routes

```
GET  /admin                      — Dashboard landing page
GET  /admin/semesters            — Semester listing
GET  /admin/semesters/new        — New semester creation (9-step wizard)
GET  /admin/semesters/[id]/edit  — Semester editor
GET  /admin/semesters/[id]/dashboard — Semester stats
GET  /admin/classes              — Class listing
GET  /admin/families             — Family listing
GET  /admin/families/[id]        — Family detail (new)
GET  /admin/payments             — Payment tracking dashboard
GET  /admin/emails               — Email broadcast listing
GET  /admin/credits              — Family account credits
```

JSON API Endpoints

```
POST   /api/register/batch-status    — Poll registration batch status
GET    /api/media                    — List media
POST   /api/media                    — Create media record
DELETE /api/media/[id]               — Delete media
POST   /api/upload-image             — Image upload (multipart/form-data)
POST   /api/webhooks/epg             — EPG payment webhook (Phase 4)
POST   /api/webhooks/resend          — Resend delivery tracking
POST   /api/webhooks/twilio          — Twilio SMS callback (Phase 5)
```

---

7. Rate Limiting (middleware.ts + utils/ratelimit.ts)

```
// Applied per IP via Upstash Redis
getWebhookLimiter()    // /api/webhooks   — 10 req/min
getUploadLimiter()     // /api/upload-*   — 5 req/min
getMediaLimiter()      // /api/media      — 20 req/min
getRegisterLimiter()   // /api/register   — 10 req/min
```

---

8. State Management

Context Providers

Provider	Storage	Responsibilities
AuthProvider	Memory	Supabase auth state, user info, sign-out
CartProvider	localStorage (2hr TTL)	Cart items, addItem(), removeItem(), clear()
RegistrationProvider	sessionStorage	Multi-step wizard: email, participants, form data, batch ID
SemesterDataProvider	Memory (hydrated from server)	Semester + sessions data during browsing

Semester Editor State (SemesterForm.tsx)

```
const [state, dispatch] = useReducer(semesterReducer, initialState)

// Dirty tracking — warns on navigation if unsaved
useEffect(() => {
  if (isDirty) window.addEventListener('beforeunload', handleBeforeUnload)
}, [isDirty])

// On step change:
await persistSemesterDraft(state)
navigate(nextStep)
```

---

9. Key Utilities

Tuition Engine (utils/tuitionEngine.ts)

Centralized pricing calculation supporting:
- **Progressive discounts:** Per additional weekly class (e.g., 2nd class 5% off base)
- **Special programs:** Fixed fees (Technique, Pointe, Competition, Early Childhood)
- **Weekly class limits:** Junior max 3/week, Senior max 6/week
- **Costume fees:** Senior $65/class, Junior $55/class (exempt: technique, pointe, competition)
- **Registration fee:** $40/child
- **Video fees:** $15/registrant (senior only)
- **Family discount:** $50 off when 2+ dancers register

Email Token Replacement (utils/resolveEmailVariables.ts)

```
resolveEmailVariables(htmlBody, {
  parent_name: "Jane Doe",
  student_name: "Emma Doe",
  semester_name: "Spring 2026",
  session_name: "Ballet 101",
  hold_until_datetime: "2026-03-17 14:00:00",
  accept_link: "https://...",
  unsubscribe_link: "https://...",
})
// Unknown tokens preserved as {{key}} — safe for forward compatibility
```

EPG API Client (utils/payment/epg.ts)

HTTP Basic auth against Elavon REST API:

```
createEpgOrder()              // Create order with amount + batchId
createEpgPaymentSession()     // Get hosted checkout URL
fetchEpgTransaction()         // Query transaction status
createEpgShopper()            // Create customer profile
createEpgStoredCard()         // Save card for recurring charges
createEpgStoredAchPayment()   // Save ACH for recurring charges
```

---

10. Major Workflows

User Registration Flow

```
1. User visits /semester/[id]
   ├─ Server fetches semester + sessions
   └─ Renders SessionGrid with CartDrawer

2. User adds sessions to cart
   ├─ CartProvider updates localStorage
   └─ TTL: now + 2 hours

3. User clicks Checkout → /register
   ├─ Step 1: Email entry
   ├─ Step 2: Participants (select/create dancers, assign to sessions)
   ├─ Step 3: Dynamic form (semester-specific questions)
   ├─ Step 4: Payment (EPG hosted checkout)
   └─ Step 5: Confirmation

4. On payment success
   ├─ Create RegistrationBatch (status='confirmed')
   ├─ Create Registration rows (1 per dancer-session pair)
   ├─ Send confirmation email
   └─ Clear cart + redirect to /register/confirmation
```

Admin Semester Creation (9-step wizard)

```
Step 1: Details (name, description, dates)
Step 2: Sessions (day, time, capacity, instructor)
Step 3: Session Groups (bundling)
Step 4: Payment Plan (pay-in-full vs installments)
Step 5: Discounts (link global discount rules)
Step 6: Registration Form (dynamic field builder)
Step 7: Confirmation Email (TipTap rich text)
Step 8: Waitlist Settings (enabled, expiry, stop window)
Step 9: Review + Publish

Each step calls persistSemesterDraft() server action which:
├─ Updates all semester fields
├─ Syncs sessions (deletes removed, upserts existing)
├─ Syncs session groups
├─ Syncs payment plan
└─ Syncs discounts

On publish:
├─ Changes status to 'published'
├─ Locks further editing (DB triggers)
└─ Semester becomes visible to users
```

Email Broadcast

```
1. Admin creates email (status='draft')
   ├─ Fills subject, sender, body (TipTap)
   └─ Selects recipients: semester | session | manual

2. Admin schedules or sends immediately
   ├─ snapshot recipients → email_recipients table
   └─ status = 'scheduled' | 'sending'

3. pg_cron every 1min triggers process-scheduled-emails edge fn
   ├─ resolveEmailVariables() — token replacement
   ├─ prepareEmailHtml() — Outlook normalization
   ├─ resend.emails.send() — dispatch
   └─ Upsert email_deliveries with resend_message_id

4. Resend webhook → /api/webhooks/resend
   └─ Updates email_deliveries.status + timestamps
```

Waitlist Automation

```
pg_cron every 1 minute → process-waitlist edge function

For each session with capacity:
├─ COUNT(active registrations) < capacity?
├─ No active 'invited' entry?
├─ Within stopDaysBeforeClose window?
└─ Fetch next waiting entry (ORDER BY position)

    → Update entry status='invited'
    → Send invitation email with token
       ├─ Token: SHA256(invite_token)
       ├─ Link: /waitlist/accept/[token]
       └─ Expires: now + inviteExpiryHours

User visits /waitlist/accept/[token]
├─ Validate token
├─ Create registration (status='pending')
├─ Set 30-min hold (hold_expires_at)
└─ Update entry status='accepted'

If hold expires:
├─ pg_cron releases pending registration
├─ Capacity released
└─ Next waitlist entry invited
```

EPG Payment Processing (Phase 4)

```
1. User completes registration → clicks "Pay Now"

2. Backend:
   ├─ calculateTotalAmount()
   ├─ Create RegistrationBatch (status='pending')
   ├─ Create Payment record (state='initiated')
   └─ createEpgOrder() + createEpgPaymentSession()
      → Returns hosted checkout URL

3. User fills in card/ACH on EPG hosted page
   └─ EPG redirects to /register/confirmation?batch_id=...

4. EPG sends webhook → /api/webhooks/epg
   ├─ Validate HTTP Basic auth (constant-time compare)
   ├─ Fetch full transaction from EPG (never trust webhook body alone)
   ├─ Map event type → payment state
   ├─ Update Payment record (state, raw_transaction)
   │
   └─ If captured/settled:
       ├─ Create BatchPaymentInstallment rows (if auto-pay)
       ├─ Update RegistrationBatch status='confirmed'
       └─ Create Registration rows + send confirmation email

5. Auto-pay installments (Phase 4)
   ├─ Stored card/ACH used for future charges
   ├─ Charges tracked in batch_payment_installments + installment_charges
   └─ Admin can manually mark installment paid via /admin/payments
```

---

11. Environment Variables

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Email (Resend)
RESEND_API_KEY=
RESEND_FROM_EMAIL=

# Payment (Elavon EPG)
EPG_BASE_URL=                        # UAT or production
EPG_MERCHANT_ALIAS=
EPG_SECRET_KEY=
EPG_WEBHOOK_USERNAME=
EPG_WEBHOOK_PASSWORD=

# SMS (Twilio — Phase 5)
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_PHONE_NUMBER=

# Rate limiting (Upstash)
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=

# Site
NEXT_PUBLIC_BASE_URL=
SITE_URL=
```

12. Key Architectural Patterns

Server Actions (Next.js App Router)

All mutations via server actions — no REST API for admin or registration:

```
"use server"

export async function persistSemesterDraft(state) { ... }
export async function publishSemester(id) { ... }
export async function sendEmailNow(emailId) { ... }
export async function markInstallmentPaid(installmentId) { ... }
```

Form Management

- **React Hook Form + Zod** for registration steps
- **useReducer** for multi-step admin forms (SemesterForm, EmailForm)
- **TipTap** for rich text email body editing
- **Dynamic field builder** for semester registration forms (JSONB schema)

Database Access Patterns

- **Server components** call Supabase directly (with `createClient()` from server utils)
- **Server actions** use admin client (`createAdminClient()`) for privileged mutations
- **Client components** use Supabase browser client for real-time or user-scoped queries
- **No ORM** — raw Supabase query builder throughout

Supabase Clients

```
// utils/supabase/server.ts — for server components + server actions
createClient()           // Uses anon key + cookie auth

// utils/supabase/admin.ts — for privileged server actions
createAdminClient()      // Uses service role key (bypasses RLS)

// utils/supabase/client.ts — for client components
createBrowserClient()    // Uses anon key
```

13. Key Files Reference

File	Purpose	Approx. Lines
types/index.ts	All domain types — single source of truth	~1827
types/public.ts	Public-safe type subset	~228
app/admin/layout.tsx	Admin shell + auth guard + sidebar nav	~222
app/admin/semesters/SemesterForm.tsx	Multi-step editor orchestrator	~500
app/admin/page.tsx	Dashboard landing page	~400
utils/tuitionEngine.ts	Pricing calculation engine	~250
utils/resolveEmailVariables.ts	Token replacement utility	~100
utils/payment/epg.ts	EPG REST API client	~500
queries/admin/index.ts	Admin data queries	~500
queries/admin/getFamilyDetail.ts	Family detail query (new)	~100
supabase/migrations/20260302000000_baseline.sql	Core schema	3123
supabase/functions/process-waitlist/index.ts	Waitlist automation cron	~300
supabase/functions/send-email-broadcast/index.ts	Batch email sender	~200
middleware.ts	Auth + rate limiting	~61

14. Known Issues & Risks

Critical

1. **Email status marked `sent` before dispatch** — Silent send failure possible
2. **Broadcast emails send empty `student_name` / `semester_name`** — Token substitution gap

High

1. Discount computation location unclear (potential client-side manipulation risk)
2. Payment confirmation mechanism needs audit (registration finalization safety)

Medium

1. Non-atomic waitlist invite (crash leaves entry stuck as `invited`)
2. TOCTOU race on waitlist capacity check
3. Cart holds live session references — no price snapshot taken at add-to-cart time
4. No optimistic locking for concurrent admin semester editing

Low

1. DB trigger bug: `RETURN NEW` on DELETE trigger
2. Three divergent token-replacement implementations across codebase
3. Confirmation email JSONB has no version history
4. Waitlist cron is sequential — bottleneck at scale

---

15. Future Phases

Phase	Description	Status
1	Class catalog + session architecture	Complete
2	Semester pricing engine	Complete
3	Tuition engine + fee config	Complete
4	EPG payment integration (Elavon)	Complete (2026-03-16)
5	SMS notifications (Twilio)	Planned
6	Competition program enhancements	Planned
7	Advanced reporting & analytics	Planned
8	Student app / mobile integration	Planned

---

16. Deployment

- **App:** Next.js 16 deployed to **Vercel**
- **Database: Supabase** hosted PostgreSQL (with pg_cron for scheduled jobs)
- **Edge Functions: Supabase Functions** (Deno runtime) for waitlist + email crons
- **Email: Resend** API for transactional + broadcast email
- **Payments: Elavon EPG** (hosted checkout + stored payment methods)
- **CDN:** Vercel Edge Network
- **Redis: Upstash** for rate limiting
- **SMS: Twilio** (Phase 5)